

TransferLearning

```
In [18]: %matplotlib inline
```

```
In [19]: from torchvision import transforms
import torchvision.transforms as transforms
import os
```

```
In [20]: batch_size = 32 # 指定batch_size
IMSIZE = 224
```

数据准备

```
In [21]: # 测试集无需数据增强，将读入的图片resize后，转换成tensor
val_transform = transforms.Compose([
    transforms.Resize((IMSIZE, IMSIZE)),
    transforms.ToTensor()
])
```

```
In [22]: # 训练集进行数据增强
train_transform = transforms.Compose([
    transforms.Resize((IMSIZE, IMSIZE)), # Resize成规定尺寸
    transforms.RandomCrop(IMSIZE, padding=8), #随机裁剪
    transforms.RandomHorizontalFlip(), #水平翻转
    transforms.ToTensor()
])
```

数据加载

```
In [23]: import tarfile
import torchvision.datasets.utils as utils
from torchvision import datasets
from torch.utils.data import DataLoader

data_dir = './data'
os.makedirs(data_dir, exist_ok=True)
tar_path = os.path.join(data_dir, 'cifar-100-python.tar.gz')
extract_dir = os.path.join(data_dir, 'cifar-100-python')

# 如果存在压缩包但未解压，则自动解压
if os.path.exists(tar_path) and not os.path.exists(extract_dir):
    with tarfile.open(tar_path, 'r:gz') as tar:
        tar.extractall(path=data_dir)

train_dataset = datasets.CIFAR100(root='./data', train=True, download=False, transform=train_transform)
test_dataset = datasets.CIFAR100(root='./data', train=False, download=False, transform=val_transform)

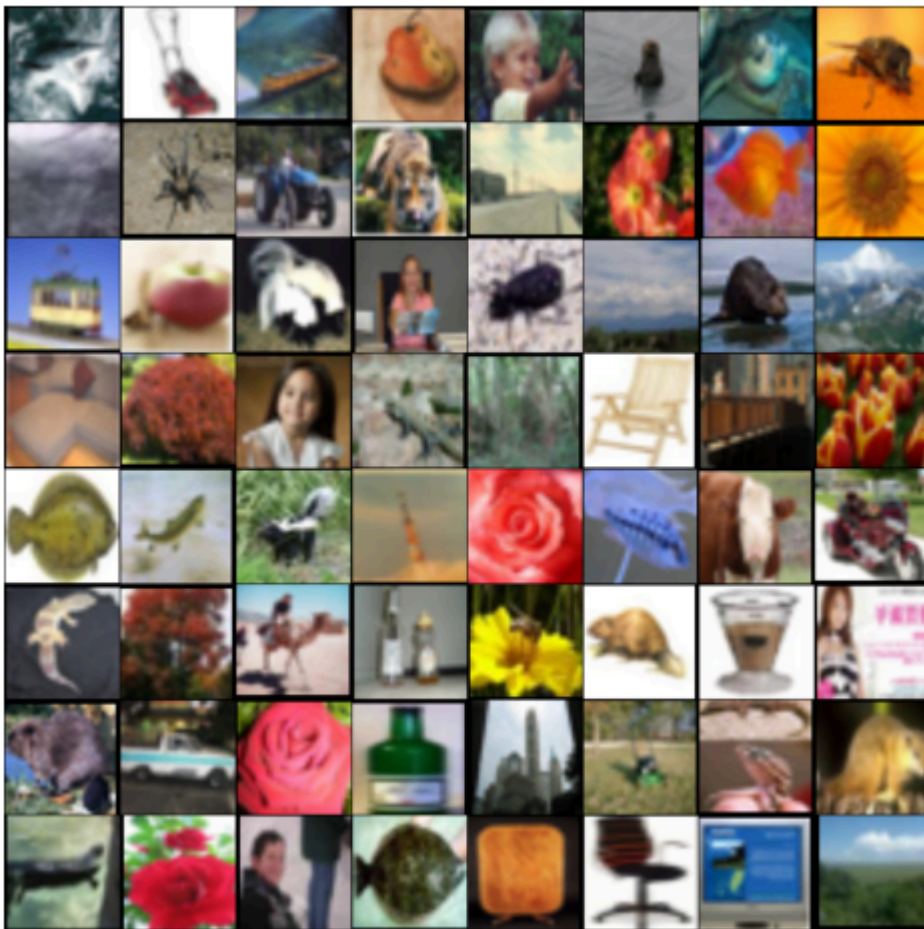
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True, num_workers=4)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False, num_workers=4)
```

数据展示

```
In [24]: from matplotlib import pyplot as plt
          from torchvision.utils import make_grid
          images, labels = next(iter(train_loader)) # 获取训练集第一个批次中的图片及相应标签
          print(images.shape)
          print(labels.shape)
          plt.figure(figsize=(12, 6)) # 设置画布大小
          plt.axis('off') # 隐藏坐标轴
          plt.imshow(make_grid(images, nrow=8).permute((1, 2, 0))) # make_grid函数把多张图片
          plt.show()
```

```
torch.Size([64, 3, 224, 224])
```

```
torch.Size([64])
```



模型训练，测试所需函数

```
In [25]: import torch
import torch.nn.functional as F
import time
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# 计算准确率
def accuracy(outputs, labels):
    preds = torch.argmax(outputs, dim=1)
```

```

    return (preds == labels).float().mean().item()
# 验证集上评估模型
def validate(model, val_loader):
    model.eval()
    val_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():
        for inputs, labels in val_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs) # 前向传播
            loss = F.cross_entropy(outputs, labels) # 计算损失
            val_loss += loss.item() # 累加损失
            preds = torch.argmax(outputs, dim=1) # 预测类别
            correct += (preds == labels).sum().item() # 计算正确预测的数量
            total += labels.size(0) # 累加样本总数
    val_loss /= len(val_loader) # 平均损失
    val_acc = correct / total
    return val_loss, val_acc
# 打印日志函数
def print_log(epoch, train_time, train_loss, train_acc, val_loss, val_acc, epoch)
    model_acc = val_acc * 100
    model_score = 20 * val_acc / 0.8
    print(f"Epoch [{epoch+1}/{epochs}], time: {train_time:.2f}s, "
          f"loss: {train_loss:.4f}, acc: {train_acc:.4f}, "
          f"val_loss: {val_loss:.4f}, val_acc: {val_acc:.4f}, "
          f"model_acc: {model_acc:.2f}%, model_score: {model_score:.4f}")

def train(model, optimizer, train_loader, val_loader, epochs=10):
    # 记录训练过程指标
    train_losses = []
    val_accuracies = []

    for epoch in range(epochs):
        model.train()
        train_loss = 0.0
        correct = 0
        total = 0
        start = time.time()

        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(inputs) # 前向传播
            loss = F.cross_entropy(outputs, labels) # 计算损失
            loss.backward() # 反向传播
            optimizer.step() # 更新参数
            train_loss += loss.item()
            preds = torch.argmax(outputs, dim=1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)

        train_loss /= len(train_loader) # 平均损失
        train_acc = correct / total
        val_loss, val_acc = validate(model, val_loader) # 在验证集上评估模型
        train_time = time.time() - start

    # 打印日志
    print_log(epoch, train_time, train_loss, train_acc, val_loss, val_acc, e

```

```

        # 保存每轮指标
        train_losses.append(train_loss)
        val_accuracies.append(val_acc)

    # 绘制曲线
    plt.figure(figsize=(10, 4))

    plt.subplot(1, 2, 1)
    plt.plot(range(1, epochs + 1), train_losses, marker='o', label='Train Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Training Loss Curve')
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(range(1, epochs + 1), val_accuracies, marker='s', color='orange', label='Validation Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.title('Validation Accuracy Curve')
    plt.legend()

    plt.tight_layout()
    plt.show(block=True)

```

迁移模型

```

In [26]: from torchvision.models import vit_b_16
import torch.nn as nn
import torch

# ViT-B/16 预训练模型, pretrained=True 表示加载在ImageNet-1K上预训练好的权重
vit_model = vit_b_16(pretrained=True)

# 修改分类层, 原始的 ViT-B/16 最后一层是一个 Linear(768 -> 1000), 对应 ImageNet 1000 个类别
# vit_model.heads.head.in_features 一般是 768, 对应 Transformer 输出的特征维度
# 把它改成 Linear(768 -> 100), 因为 CIFAR-100 有 100 个类别
vit_model.heads.head = nn.Linear(vit_model.heads.head.in_features, 100)

# 放入GPU
vit_model = vit_model.cuda()

```

```

/root/miniconda3/envs/align-anything/lib/python3.11/site-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.
  warnings.warn(
/root/miniconda3/envs/align-anything/lib/python3.11/site-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=ViT_B_16_Weights.IMAGENET1K_V1`. You can also use `weights=ViT_B_16_Weights.DEFAULT` to get the most up-to-date weights.
  warnings.warn(msg)

```

展示模型信息

```

In [27]: from torchinfo import summary
vit_model = vit_model.cuda().eval()

```

```
summary(  
  vit_model,  
  input_size=(1, 3, 224, 224),    # 批量大小=1  
  col_names=["input_size", "output_size", "num_params"],  
  verbose=0  
)
```

```

Out[27]: =====
=====
Layer (type:depth-idx)      Input Shape      Output
Shape                      Param #
=====
VisionTransformer          [1, 3, 224, 224] [1, 10
0]                          768
├─Conv2d: 1-1              [1, 3, 224, 224] [1, 76
8, 14, 14]                590,592
├─Encoder: 1-2             [1, 197, 768]    [1, 19
7, 768]                  151,296
|   └─Dropout: 2-1         [1, 197, 768]    [1, 19
7, 768]                  --
|   └─Sequential: 2-2      [1, 197, 768]    [1, 19
7, 768]                  --
|   |   └─EncoderBlock: 3-1 [1, 197, 768]    [1, 19
7, 768]                  7,087,872
|   |   └─EncoderBlock: 3-2 [1, 197, 768]    [1, 19
7, 768]                  7,087,872
|   |   └─EncoderBlock: 3-3 [1, 197, 768]    [1, 19
7, 768]                  7,087,872
|   |   └─EncoderBlock: 3-4 [1, 197, 768]    [1, 19
7, 768]                  7,087,872
|   |   └─EncoderBlock: 3-5 [1, 197, 768]    [1, 19
7, 768]                  7,087,872
|   |   └─EncoderBlock: 3-6 [1, 197, 768]    [1, 19
7, 768]                  7,087,872
|   |   └─EncoderBlock: 3-7 [1, 197, 768]    [1, 19
7, 768]                  7,087,872
|   |   └─EncoderBlock: 3-8 [1, 197, 768]    [1, 19
7, 768]                  7,087,872
|   |   └─EncoderBlock: 3-9 [1, 197, 768]    [1, 19
7, 768]                  7,087,872
|   |   └─EncoderBlock: 3-10 [1, 197, 768]    [1, 19
7, 768]                  7,087,872
|   |   └─EncoderBlock: 3-11 [1, 197, 768]    [1, 19
7, 768]                  7,087,872
|   |   └─EncoderBlock: 3-12 [1, 197, 768]    [1, 19
7, 768]                  7,087,872
|   └─LayerNorm: 2-3       [1, 197, 768]    [1, 19
7, 768]                  1,536
├─Sequential: 1-3         [1, 768]          [1, 10
0]                          --
|   └─Linear: 2-4         [1, 768]          [1, 10
0]                          76,900
=====
Total params: 85,875,556
Trainable params: 85,875,556
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 172.54
=====
Input size (MB): 0.60
Forward/backward pass size (MB): 104.09
Params size (MB): 229.50
Estimated Total Size (MB): 334.19
=====
=====

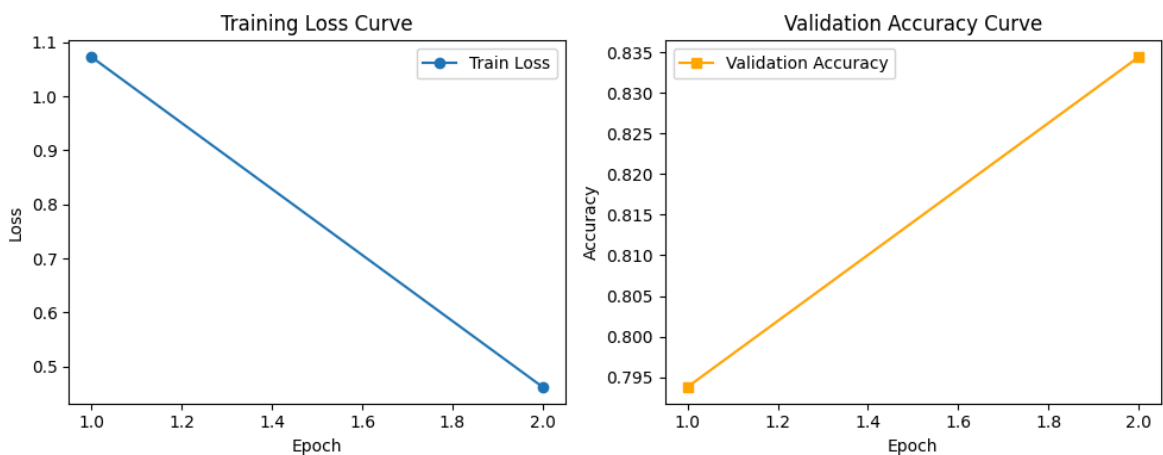
```

开始训练

```
In [28]: # 在加载的模型基础上使用新数据集进行微调, 实现迁移学习
lr = 1e-4
optimizer = torch.optim.Adam(vit_model.parameters(), lr=lr)
epochs = 2
history = train(vit_model, optimizer, train_loader, test_loader, epochs=epochs)
```

Epoch [1/2], time: 164.45s, loss: 1.0729, acc: 0.7233, val_loss: 0.7064, v
model_score: 19.8450

Epoch [2/2], time: 164.43s, loss: 0.4618, acc: 0.8598, val_loss: 0.5744, v
model_score: 20.8600



进行测试

```
In [29]: import numpy as np
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
vit_model.eval() # 关闭dropout/batchnorm

classes = train_dataset.classes

indices = np.random.choice(len(test_dataset), 5, replace=False)
fig, axes = plt.subplots(1, 5, figsize=(18, 4))

for i, idx in enumerate(indices):
    img, label = test_dataset[idx]

    # 展示图像
    axes[i].imshow(np.transpose(img, (1, 2, 0))) # C,H,W -> H,W,C
    axes[i].axis("off")

    # 模型预测
    with torch.no_grad():
        input_tensor = img.unsqueeze(0).to(device) # (1, 3, 224, 224)
        outputs = vit_model(input_tensor)
        probs = F.softmax(outputs, dim=1)
        prob, pred = torch.max(probs, 1) # 从概率中取最大值和索引
        prob = prob.item()
        pred = pred.item()
```

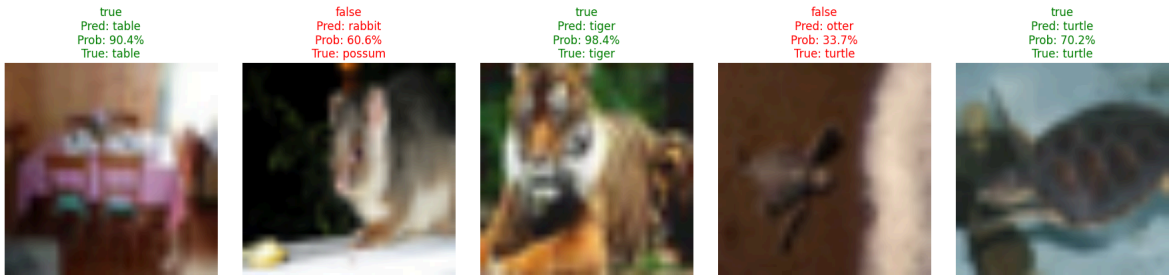
```

# 比对预测结果
true_label = classes[label]
pred_label = classes[pred]
correct = (pred == label)

# 设置标题
color = "green" if correct else "red"
result_text = "true" if correct else "false"
axes[i].set_title(f"{result_text}\nPred: {pred_label}\nProb: {prob*100:.1f}%")

plt.tight_layout()
plt.show()

```



像 table、tiger、turtle 这类分类正确且都有较高置信度，说明模型成功学习到了较稳定的语义特征。不过，也能看出在细粒度动物类别上仍存在混淆，比如 possum 被误判为 rabbit、turtle 被识别成 otter，说明模型在处理形态相近或纹理相似的类别时判别能力不足。这类错误通常源于特征分布过于接近、样本数量不平衡或图像背景干扰较大。当不同类别在纹理、颜色或形态上相似时，模型容易在判别边界上产生模糊。这个从判别错误的图片其概率最高的答案没有其他判别正确答案的概率高就可以看出来，在这种情况下，模型对于各种类别判别也不是很确定。可以通过引入更好的数据增强策略或者说分层的微调让模型保持通用能力的同时提升细粒度区分能力。